

SecurAx Scanner Engine

Mise à jour	Inclusion des sous-moteurs (ZAP, Nuclei, Nikto, Semgrep, Gitleaks) et améliorations.
Sujet	Périmètre d'action, limites techniques et voies de dépassement des limites.
Langue	Français
Date	Juin 2026

1. SCANNER DAST MULTI-MOTEURS (dast_scanner.py)

1.1 Capacités et Périmètre d'Action

Orchestrateur d'analyse dynamique des applications web (Dynamic Application Security Testing) utilisant 3 moteurs en parallèle : OWASP ZAP, Nuclei, et Nikto.

Fonctionnalité	Détail de l'analyse
OWASP ZAP	Spidering actif et injection dynamique (XSS, SQLi, CSRF). Démarre un démon et pilote l'API REST.
Nuclei (CLI/Cloud)	Plus de 9000 templates YAML (CVEs, expositions, default-login). S'exécute localement ou via l'API Cloud ProjectDiscovery.
Nikto	Vérifications de configuration des serveurs web et fichiers par défaut obsolètes. Utilise Perl, la CLI ou Docker en fallback.
SSRF Protection	Bloque par défaut les adresses IP privées/locales pour éviter que le scanner lui-même ne soit utilisé pour du SSRF.

1.2 Méthodologie et Outils

1	Exécution Concurrente	Utilise ThreadPoolExecutor pour lancer ZAP, Nuclei et Nikto en même temps, maximisant la vitesse.
2	Fusion (Deduplication)	Les vulnérabilités trouvées par plusieurs outils sont fusionnées en une seule alerte avec une 'Confiance' (Confidence) augmentée.

1.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Temps d'exécution massif	Le Spidering de ZAP combiné aux milliers de templates Nuclei génère un trafic massif, souvent confondu avec un DDoS par les WAF.
Authentification Complexe	Incapable de scanner derrière des portails SSO, MFA, ou Captchas sans macros d'authentification lourdes ZEST.

Limite (Ce qu'il ne fait pas)	Explication technique
Boîte Noire absolue	Le scanner ne voit pas le code ; il déduit les failles depuis le comportement HTTP externe (erreurs 500, time-delays).

1.4 Améliorations Proposées pour Dépassez les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Bypass Authentification	Intégrer le support de Selenium/Puppeteer pour passer les portails de connexion et injecter un Header 'Authorization: Bearer' au niveau du proxy ZAP.
Throttle Intelligent	Implémenter une limitation de taux (Rate-Limiting) dynamique qui ralentit l'envoi de requêtes si le serveur renvoie des codes 429 (Too Many Requests).
Intégration IAST	Coupler le DAST avec un agent logiciel interne au serveur (IAST) pour lier la requête malveillante directement à la ligne de code source visée.

2. SCANNER SAST MULTI-LANGAGES (sast_scanner.py)

2.1 Capacités et Périmètre d'Action

Orchestrateur d'analyse statique du code source intégrant 4 moteurs parallèles : Bandit, Semgrep, Gitleaks, et un scanner Regex natif (Built-in).

Fonctionnalité	Détail de l'analyse
Semgrep	Scanner polyglotte (Python, JS, Java, Go, etc.) utilisant les règles OWASP Top 10 et CWE Top 25 extraites de semgrep.dev.
Bandit	Linter de sécurité dédié spécifiquement à Python, soutenu par la PyCQA.
Gitleaks	Détection de plus de 150 types de secrets hardcodés (clés AWS, tokens GitHub, clés privées Stripe).
Scanner Natif	File de secours interne en Python pur basé sur des expressions régulières pour détecter <code>eval()</code> , <code>os.system()</code> et les secrets critiques, même si aucun outil n'est installé.

2.2 Méthodologie et Outils

1	Arbre Syntaxique Abstrait (AST)	Bandit et Semgrep compilent le code en AST pour comprendre la sémantique (ex: repérer si un input est passé à <code>eval()</code> sans validation).
2	Isolement Sécurisé (Chroot/Temp)	Décompresse les archives ZIP dans un dossier temporaire chrooté avec vérification contre les 'ZIP Bombs' et les attaques de Directory Traversal (<code>../</code>).

2.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Faux Positifs Fréquents	S'alarme sur des noms de variables (ex: <code>'test_password'</code>) ou du code inactif (dossiers <code>/tests/</code>), abaissant le ratio signal/bruit.
Ignorance du Flux d'Exécution	Analyse les fichiers individuellement, mais a du mal avec le 'Taint Tracking' complexe à travers plusieurs modules et microservices.

Limite (Ce qu'il ne fait pas)	Explication technique
Manque de contexte d'exécution	Signale des failles qui peuvent être en réalité mitigées par le pare-feu réseau ou un middleware de nettoyage WAF.

2.4 Améliorations Proposées pour Dépassez les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Taint Analysis Inter-Fichiers	Évoluer de Semgrep OSS vers Semgrep Pro ou joindre un moteur de Taint Tracking Global (ex: CodeQL) pour suivre le flux de données malveillant à travers l'ensemble de l'architecture.
Filtres d'IA Générative	Utiliser un modèle LLM léger pour pré-trier les résultats et éliminer les faux positifs évidents avant de les présenter à l'utilisateur.
Contextualisation Git	Connecter le scanner au dépôt Git pour ignorer les failles vieilles de 5 ans et ne scanner que le 'diff' du code récemment modifié (Scan Diff).

3. SCANNER RÉSEAU ET PORTS (netscan_scanner.py)

3.1 Capacités et Périmètre d'Action

Cartographie du réseau et découverte de la surface d'attaque externe de l'infrastructure.

Fonctionnalité	Détail de l'analyse
Découverte de Ports	Scanne les ports TCP pour trouver les services ouverts.
Fingerprinting	Interroge les bannières pour déterminer la version du logiciel (Nmap OS detection).
Scripts NSE	Exécute des scripts Nmap pour des CVE spécifiques.

3.2 Méthodologie et Outils

1	Wrapper python-nmap	Interface le binaire système de Nmap vers Python.
2	Scan SYN furtif	Utilise TCP SYN (Half-open) pour rester discret.

3.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Bloqué par Firewalls	Les IDS/IPS bloquent les scans de ports massifs.
Pas de contexte HTTP	Connaît le port, mais pas l'arborescence web sous-jacente.

3.4 Améliorations Proposées pour Dépassez les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Rotation d'IP	Utiliser un pool d'adresses IP proxies (Tor/VPN) pour distribuer le scan et échapper aux blocages par adresse IP unique.
Timing Évasif	Ajouter l'option -T2 (Polite) ou -T1 (Sneaky) pour ralentir drastiquement l'envoi des paquets et passer sous le radar des IDS, bien que cela allonge la durée du scan.

4. SCANNER SSL / TLS (ssl_scanner.py)

4.1 Capacités et Périmètre d'Action

Audit cryptographique de la couche de transport sécurisée (HTTPS/TLS).

Fonctionnalité	Détail de l'analyse
Expiration	Alerte sur les certificats proches de l'expiration.
Protocoles	Détecte SSLv2/v3 et TLS 1.0/1.1.
Ciphers Faibles	Vérifie l'absence de Forward Secrecy et les algorithmes dépréciés.

4.2 Méthodologie et Outils

1	Sockets natifs	Utilise `ssl` en Python sans outils externes.
2	Parsing ASN.1	Extrait directement les dates et métadonnées du certificat X.509.

4.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Vulnérabilités Côté Serveur	Ne détecte pas Heartbleed ou CCS Injection (qui requièrent des payloads spécifiques malformés).
Limitations de port	Ne scanne par défaut que le port 443, ignorant les services TLS sur des ports non standards (ex: 8443, 465, 993).

4.4 Améliorations Proposées pour Dépasser les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Bibliothèque Dédiée	Intégrer testssl.sh ou SSLyze (via subprocess) pour tester activement les failles cryptographiques avancées (POODLE, BEAST, ROBOT, Heartbleed).
Scan Multi-Ports	Coupler cet outil avec le Network Scanner pour lancer automatiquement l'audit SSL sur TOUS les ports découverts comme utilisant du SSL, pas seulement le 443.

5. SCANNER DE DÉPENDANCES (dep_scanner.py)

5.1 Capacités et Périmètre d'Action

Analyse de la Supply Chain (SCA).

Fonctionnalité	Détail de l'analyse
SCA SBOM	Identifie les paquets tiers depuis requirements.txt, package.json, etc.
Mapping CVE	Croise avec OSV.dev pour trouver les failles connues.
Détection de Malwares	Repère les dépendances abandonnées ou malveillantes.

5.2 Méthodologie et Outils

1	API OSV.dev	Interroge la base Google OSV en temps réel.
2	Parsing Regex/JSON	Extrait les versions exactes depuis les lockfiles.

5.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Faux Positifs structurels	Signale une librairie vulnérable même si le composant défaillant n'est jamais appelé par l'application locale.
Dépendances Transitives profondes	Si aucun lockfile n'est présent (ex: juste un requirements.txt sans versions épinglées), le scanner ne peut pas deviner les sous-dépendances réelles.

5.4 Améliorations Proposées pour Dépasser les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Analyse d'Atteignabilité (Reachability)	Croiser l'arbre des dépendances avec l'AST du code SAST pour prouver que la fonction vulnérable de la librairie est effectivement importée et appelée dans le code (Call Graph).
Auto-Génération de SBOM	Utiliser un outil comme Syft pour générer automatiquement l'arbre des dépendances transitives complètes même en l'absence de fichier 'lock'.

6. SCANNER WEB DE BASE (web_scanner.py)

6.1 Capacités et Périmètre d'Action

Audit passif rapide des en-têtes et redirections.

Fonctionnalité	Détail de l'analyse
Headers	HSTS, CSP, X-Frame-Options.
Empreinte (Fingerprint)	Server, X-Powered-By.
Force HTTPS	Vérifie les redirections 301 sécurisées.

6.2 Méthodologie et Outils

1	Requête Unique	Fait un seul GET sur la page d'accueil.
---	----------------	---

6.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Pas de Crawling	Ignore 99% du site web.
Aucune action destructive	Ne valide pas la présence de failles d'injection réelles.

6.4 Améliorations Proposées pour Dépassez les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Intégration d'un Crawler Léger	Ajouter un parseur HTML BeautifulSoup qui extrait les liens de la page d'accueil et vérifie les Headers sur un échantillon de 10 sous-pages pour s'assurer que la politique CSP est globale.
Checks CORS	Ajouter des requêtes avec des 'Origin' forgées pour détecter des politiques CORS trop permissives (Access-Control-Allow-Origin: *).

7. AUDIT DE CONFIGURATION SERVEUR (server_int.py)

7.1 Capacités et Périmètre d'Action

Analyse de configuration Nginx/Apache/SSH.

Fonctionnalité	Détail de l'analyse
Masquage	Vérifie server_tokens off.
Durcissement SSH	Vérifie PermitRootLogin no.
SSL Serveur	Vérifie TLSv1.2+ dans nginx.conf.

7.2 Méthodologie et Outils

1	Recherche textuelle	Utilise Regex sur des chemins fixes (/etc/nginx/).
---	---------------------	--

7.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Statique uniquement	Ne vérifie pas la configuration chargée en RAM par le démon.
Inclusions externes	Ne suit pas les directives 'include conf.d/*'.

7.4 Améliorations Proposées pour Dépasser les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Parsing d'Arbre de Configuration	Au lieu de simples Regex, utiliser un véritable Parseur (ex: 'crossplane' pour Nginx) qui comprend la syntaxe et fusionne tous les fichiers 'include' en une configuration unique.

Axe d'Amélioration	Description Technologique
Audit via API Docker	Si les serveurs tournent sous Docker, s'interfacer avec l'API Docker pour récupérer la vraie configuration de démarrage des processus (docker inspect).

8. SONDAGE SERVEUR EXTERNE (server_ext.py)

8.1 Capacités et Périmètre d'Action

Reconnaissance initiale externe.

Fonctionnalité	Détail de l'analyse
GeoIP & ASN	Identifie l'hébergeur physique.
WAF Detection	Repère Cloudflare, Akamai, etc.
Techno stack	Devine Nginx vs Apache via l'empreinte.

8.2 Méthodologie et Outils

1	Requêtes HEAD	Minimise la bande passante.
---	---------------	-----------------------------

8.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Superficiel	Ce n'est pas un scanner de vulnérabilités.
Spoofing facile	Trompé par un admin qui modifie les headers HTTP.

8.4 Améliorations Proposées pour Dépasser les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
WAF Evasion Detection	Utiliser l'outil 'WafW00f' en arrière-plan qui envoie des requêtes intentionnellement malformées pour voir exactement quel WAF bloque (et comment il réagit).

Axe d'Amélioration	Description Technologique
Résolution Historique DNS	Interroger des services comme SecurityTrails pour découvrir la vraie IP derrière le CDN/WAF Cloudflare en regardant l'historique DNS.

9. SCANNER DNS & EMAIL SECURITY (dns_scanner.py)

9.1 Capacités et Périmètre d'Action

Vérifie SPF, DMARC, DKIM, DNSSEC et AXFR.

Fonctionnalité	Détail de l'analyse
Anti-Spoofing	Valide SPF (-all) et DMARC (quarantine/reject).
Zone Transfer	Tente AXFR sur le port TCP 53.
Signature	Vérifie DS (DNSSEC) et CAA.

9.2 Méthodologie et Outils

1	Cloudflare DoH	Contourne les pare-feu de port 53 en utilisant HTTP (requests).
2	Sockets TCP	Assemblage binaire manuel pour le test AXFR.

9.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Sélecteurs DKIM inconnus	Ne trouve que les clés DKIM standard (google, mail).
Limite de 10 lookups SPF	Ne résout pas la chaîne complète des inclusions SPF.

9.4 Améliorations Proposées pour Dépassez les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Analyseur SPF Récursif	Implémenter une machine à états qui suit et compte toutes les balises 'include' et 'a' récursivement pour vérifier que la limite stricte de 10 requêtes DNS (RFC 7208) n'est pas dépassée (SPF Flattening check).
Dictionnaire DKIM massif	Étendre la recherche de sélecteurs DKIM à un dictionnaire de 10 000 mots courants, combiné avec la vérification des enregistrements CNAME qui pointent souvent vers des services de mailing (SendGrid, Mailgun).

10. SCANNER CONTENEURS & DOCKER (docker_scanner.py)

10.1 Capacités et Périmètre d'Action

Audit statique Dockerfile / docker-compose.

Fonctionnalité	Détail de l'analyse
Root Avoidance	Alerte si aucun USER n'est défini (root par défaut).
Dangerous Mounts	Repère /var/run/docker.sock.
Supply Chain	Repère les images sans version (:latest).

10.2 Méthodologie et Outils

1	Parsing Ligne par Ligne	Logique Regex pure sans nécessiter le démon Docker.
---	-------------------------	---

10.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Aucune vue dynamique	Ne voit pas les processus exécutés au runtime.
Ignorance des dépendances internes	Ne scanne pas les vulnérabilités de l'image OS de base (ubuntu, alpine).

10.4 Améliorations Proposées pour Dépasser les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Intégration Clair/Trivy	Plutôt que de scanner le texte du Dockerfile, compiler l'image et la passer dans 'Trivy' pour analyser la nomenclature complète des librairies binaires installées dans la couche système (apt, apk).
Vérification des Signatures	Vérifier via Docker Trust (Notary) si l'image de base (FROM) est signée cryptographiquement par le fournisseur officiel.

11. SCANNER WORDPRESS

(wordpress_scanner.py)

11.1 Capacités et Périmètre d'Action

Probing de configuration et de surface d'attaque spécifique WP.

Fonctionnalité	Détail de l'analyse
User Enum	Récupère les noms d'utilisateurs via REST API.
Vecteurs DoS	Teste l'accès à xmlrpc.php et wp-cron.php.
Fichiers sensibles	Cherche wp-config.php et debug.log.

11.2 Méthodologie et Outils

1	Probing Ciblé	Sait exactement quelles URL WordPress interroger.
2	API JSON	Parse l'API REST moderne de WP.

11.3 Limites et Angles Morts

Limite (Ce qu'il ne fait pas)	Explication technique
Ignorance des Plugins	Ne vérifie pas les 50 000 plugins WP existants.
Bloqué par WAF	Wordfence bloque immédiatement ce genre de requêtes standards.

11.4 Améliorations Proposées pour Dépasser les Limites

Afin d'atténuer ou de résoudre les limites inhérentes listées ci-dessus, voici les évolutions architecturales recommandées :

Axe d'Amélioration	Description Technologique
Fuzzing de Plugins (wpscan-like)	Intégrer le téléchargement d'une base de données locale des plugins vulnérables (WPScan database) et parcourir passivement la page source HTML (wp-content/plugins/X) pour détecter leurs versions exactes sans alerter les pare-feu.
Contournement de WAF Avancé	Utiliser des techniques de manipulation des Headers (X-Forwarded-For) ou l'obfuscation d'URL pour contourner les protections basiques des plugins de sécurité comme iThemes ou Wordfence.